

Traffic simulation with the Wiedemann model

André Stebens
August 1995

Abstract

The Wiedemann model was implemented in C on a UNIX workstation. One lane traffic was simulated and the basic behaviour of following was examined. The model is described analytically by delay differential equations (ddes). The numerical solution was received by integrating with the Euler method with a small stepsize. Timesteps from 0.1 seconds down to 0.01 seconds were used compared to a timestep of 1 second used in the original Wiedemann paper. It was found, that only for the first cars a regular behaviour could be seen. For longer queues, the cars show chaotic behaviour. The motions of the cars are getting unperiodic.

Contents

1	Introduction	2
2	The Wiedemann model	3
3	Implementation	5
3.1	Routine full_speed	7
3.2	Routine follow	8
3.3	Routine approach	8
3.4	Routine emergency	9
4	Results	10
5	Outlook	13
6	Appendix	15
A	Handling of the program	15

1 Introduction

The Wiedemann model is based on a psycho-physical theory. The main idea is, that an action is only done, if there is a stimulance for doing so. The driver reacts at that time, when he can realise a difference in the distance or the velocity to the predecessor. Because the human reaction time is not infinite small but finite, the driver reacts not instantaneously. The reaction time could be handed over to the program as a parameter. For the results a reaction time of 1 second was used. That means, that the driver reacts on the situation, which was actual 1 second ago. This yields to some nasty problems. In the original paper of Wiedemann, he chooses a timestep of just this 1 second for the euler algorithm. This makes things easy, but not very accurate. The euler method is only first order accurate. This is a big source of errors, especially with such a large stepsize. Hence, a shorter stepsize has to be chosen. But now the history of each driver must be stored, because if the driver decides to brake it can not be done instantaneously, but the reaction

time later. This means, that at least the values of the calculated acceleration must be stored for each time step in the range of the reaction time.

2 The Wiedemann model

Imagine a car driver in the night. How can he decide, whether a following car is coming nearer or not? He looks different times in the rear view mirror. If he sees the lights of the car getting more and more apart he will decide, that the car is getting closer. If one thinks of angular resolution one can say, that the driver sees the following car coming closer, when the angle between left and right parts of the car increases. One can show with geometrical arguments that:

$$\frac{d\alpha}{dt} \propto threshold \propto \frac{\Delta v}{\Delta x^2}$$

This is the stimulating magnitude, which forces the driver to an action. This result is similar to mathematical car following models with suitable parameters.

What we have got so far is, that the car driver is able to realise whether the distance to the next car is increasing or decreasing. The Wiedemann model describes now how the driver behaves in different ranges. Wiedemann specifies four regimes:

- Free driving

The distance to the next car is that large, that this car has no affect on the driver. He will accelerate to his favourite velocity and keep it until the situation changes somehow.

- Following

The predecessor is driving slower than the driver would preferably drive. Usually he has got two possibilities to react: Overtake or stay behind with the appropriate velocity. Because only one lane traffic is simulated only the second possibility is valid. Each driver has got his own preferred following distance. It is assumed that the human driver is not perfect (!) and cannot estimate exactly the actual distance and velocity difference to the predecessor. Hence, he will drive a bit too fast and therefore he will come too close to the predecessor. As soon, as he realises this, he will brake and fall behind, until the distance is too large. Hence, he will accelerate again. This mechanism leads to an oscillating motion of the following car.

- Approaching

If the distance to the next car is larger than the desired following distance, the driver will accelerate. His aim is to keep the right distance and to get the same acceleration as the car in front of him. If he has reached the right distance, the regime is changing to "following".

- Emergency brake

If the distance is smaller than a driver specific security distance, the situation is critical and he will brake very strong not to crash with the following car.

Not every driver has got the same preferences. To be more realistic, a large variety of different drivers is simulated. This parameters are chosen randomly at the begining of the program:

- favourite velocity (Includes wish of the driver, capability of the car, characteristic of the road, etc.)

(Name of the variable in the program: $\text{car} \rightarrow \text{wish_v}$)

- security requirement ($\text{car} \rightarrow \text{secure}$)
- estimation property ($\text{car} \rightarrow \text{estimate}$)
- acceleration and braking possibilities of the car ($\text{car} \rightarrow \text{accel}$)
- lower limit of the control possibility of the accelerator pedal ($\text{car} \rightarrow \text{control}$)
- respond of the brake lights ($\text{car} \rightarrow \text{light}$)

All these parameters are produced randomly at the beginning of the simulation. The random numbers are gaussian with a mean value 0.5 and a standard deviation of 0.15.

3 Implementation

At each timestep the preferences are used to calculate the actual situation for each driver. To characterise the situation, Wiedemann proposed the following values (In some of the formulas he introduced a noise term. In the simulation this noise term can be switched on and off. Experiments were only done without any noise term, because it was found that the model is noisy enough. The noise term would prevent the view on the raw mechanisms.

- $d_{\max}$

This is a parameter and for all drivers the same. This is the distance, where the driver feels free and is not influenced by any other driver. Wiedemann proposed a value of 150m. He refers to measurements in America (H.C.M., 1965) and in Germany (HOEFS, 1972) which propose this value. Maybe there are newer measurements which propose a different value. In that case should the model be adjusted by changing the parameter.

- $AX(i) = carlength + 1 + 2 \cdot car(i) \rightarrow secure$

This is the desired distance of driver (i) to car (j) at velocity 0. The carlength is kept constant for all cars. Wiedemann proposed a value of 4.5m. The value can be handed over to the simulation as a parameter.

- $BX(i) = AX(i) + (1 + car(i) \rightarrow secure) \cdot \sqrt{v}$

This is the minimal following distance desired by driver (i) at approximately the same velocity of the predecessor. The formula includes that the distances do not grow with larger velocities. People tend to drive at more riscant distances at higher velocities. If car (i) is approaching car (j) the velocity of (j) is valid in this formula, otherwise it is v(i).

- $SDV(i) = ((DX - AX)/CX)^2$ with the auxiliary variable

$$CX(i) = 25 \cdot (1 + car(i) \rightarrow secure + car(i) \rightarrow estimate)$$

SDV denotes a perception treshold at large distances at which the driver can re-alise velocity differences. The higher the security requirement and the higher the estimation capability of the driver, the smaller is the realised velocity difference.

- $SDX(i) = AX + EX \cdot (BX - AX)$ with the auxiliary variable

$$EX = 2 - car(i) \rightarrow estimate(+noise)$$

This is the upper boundary for the drift in the following process. The value of SDX is fluctuating between $1.5 \cdot BX$ and $2.5 \cdot BX$ (TODOSIEV, 1963 and HOEFS, 1972).

Wiedemann argues, that this value is fluctuating for each driver as well and puts therefore the noise term into the calculation. If the estimation capability is large, than is EX and hence SDX small.

- $CLDV = SDV \cdot EX^2$

This term specifies the perception treshold for small velocities at small, decreasing distances. The value is larger than that for SDV. The same driver characteristics are affecting this value, therefore the auxiliary variable EX can be used for calculating this variable as well.

- $OPDV = CLDV \cdot (-1(-2 \cdot noise))$

Again a term for small velocities but this time for small, increasing distances. The fluctuation is still larger than CLDV.

The structure of the formula system can be summarised in the following diagram. The $\Delta v - \Delta x$ diagrams offer the most information to characterise the system.

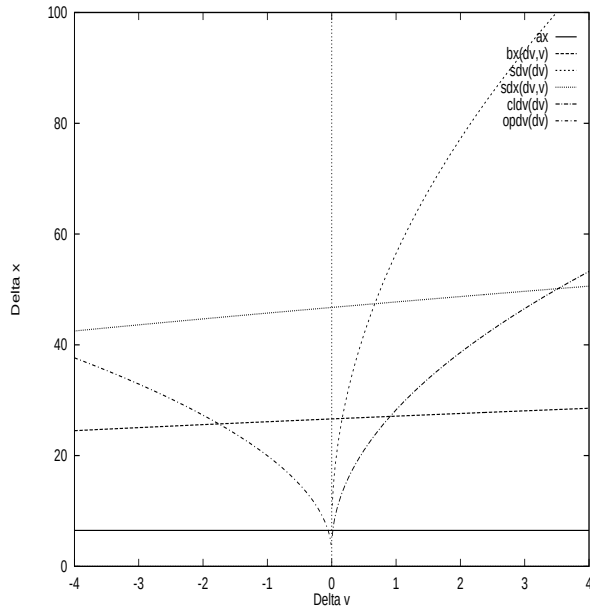


Figure 1: $\Delta v - \Delta x$ diagram for $v=20\text{m/s}$.

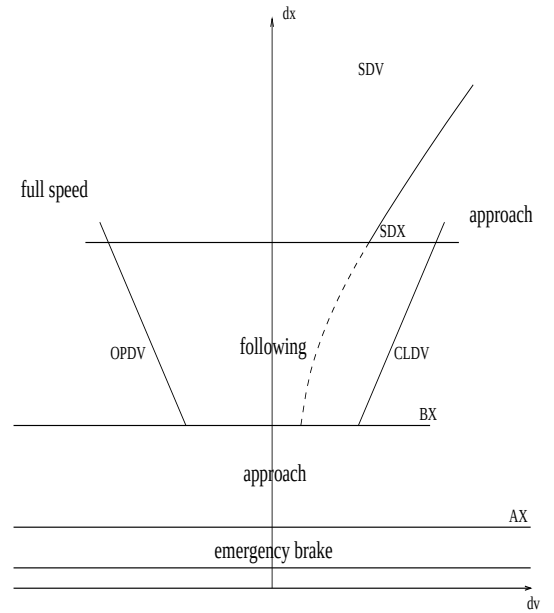


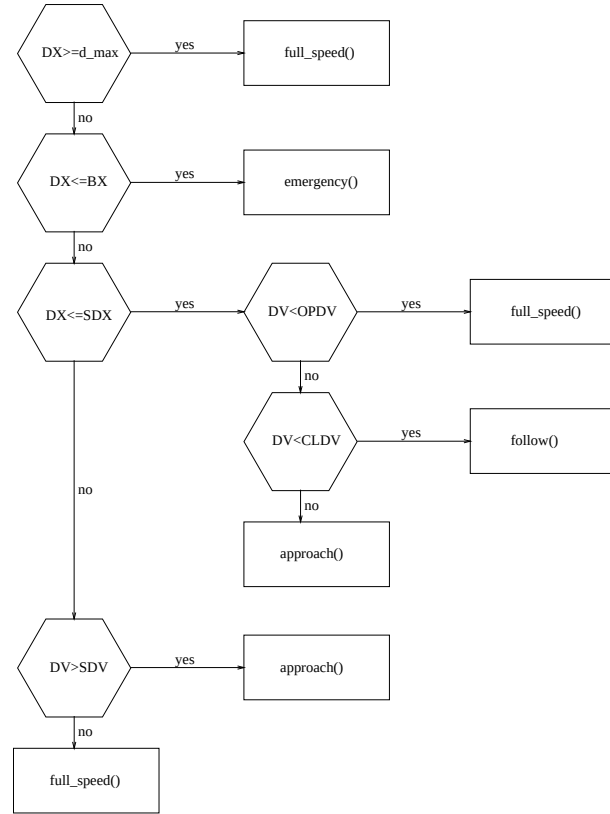
Figure 2: Schematic overview

The implementation of the classifying of the regime in the program takes place by the following if-statements:

```

if(DX>=d_max)
    b=full_speed();
else{
    if(DX<=BX)
        b=emergency();
    else{
        if(DX<=SDX)
            if(DV<OPDV)
                b=full_speed();
            else
                if(DV<CLDV)
                    b=follow();
                else
                    b=approach();
        else
            if(DV>SDV)
                b=approach();
            else
                b=full_speed();
    }
}

```



3.1 Routine full_speed

This routine is used, if the driver is not influenced by any other driver. He will accelerate, until he reaches his preferred velocity. The acceleration is not linear. It is getting less with larger velocity. In addition is the acceleration dependent on the driver specific parameter $\text{car}(i) \rightarrow \text{accel}$. In the program, the calculation is done by:

```

bmax = (0.2 + 0.8*car->accel)*(7-sqrt(car->v));
if(DX<=2*BX) b = bmax * (DX-BX)/BX; else b = bmax;
if(b<bnull) b = bnull;
if(car->v[0] > car->wish_v) b = -bnull;

```

The second line is valid, if the driver has just finished a following process. The acceleration should be smaller as if he would have accelerated from a smaller velocity. After the range $2 \cdot BX$ he is in full acceleration. The acceleration never vanishes. The minimal value is

given by `bnull` and is driver dependent. The value is given by:

```
bnull = 0.2*(car->accel+noise);
```

If the velocity is larger than the desired velocity, the acceleration is set to `-bnull`. Hence, the velocity is never really constant but fluctuating about a constant value.

3.2 Routine follow

This routine is quite simple. As mentioned before, the acceleration is never exactly zero. It is assumed, that a driver keeps his acceleration in the following process. That means for this routine, that the sign of the acceleration one timestep before is valid (should be stored in a history array or ring list). The magnitude is `bnull`. This strategy yields unavoidable to a change of the regime. Either the car is coming too close to his predecessor ($\rightarrow$ approach) or too far away ($\rightarrow$ full_speed). The result is an oscillating motion of the following car. It is shown later, that for the first following car this oscillation is periodic and shows a close orbit in the $dv-dx$ -diagram. But in a queue just a few cars later the motion is not periodic any longer and shows chaotic behaviour.

3.3 Routine approach

In this routine, the driver tries to adjust his velocity to that of the predecessor and stay in the preferred distance (`BX`) behind. If the velocity of the predecessor is constant, the problem is not too hard. But if this velocity is changing, the behaviour can be quite complicated. One has to keep in mind, that delay differential equations are simulated or in other words, that the driver has got a finite reaction time. Hence he adjusts his velocity to that of his predecessor 1 second ago. Wiedemann proposed as a formula for the acceleration:

```
b(i,t) = 0.5*DV*DV/(BX-DX) + b(j,t-reaction_time);
```

There are a few restrictions according to this formula: The acceleration of the predecessor is only put into account, if this acceleration is recognizable to driver (`i`). The limit of recognition is given by:

```
br = -0.5 - 1.5*(pre->accel);
```

If the acceleration of (`j`) is smaller than this value and the distance is small enough ($DX < 2 \cdot BX$) the formula shown above is used to calculate the actual acceleration of (`i`), otherwise the term `b(j,t-reaction_time)` at the end is omitted. Wiedemann did not

say anything about measurements of recognition of acceleration changes. I think, one can doubt that changes in acceleration are really recognizable.

It should not be kept in secret that this routine includes other problems. The aim of driver (i) is to reach the distance BX. But at this distance the formula for the acceleration has got a singularity. Hence the acceleration goes to infinity, when the car reaches the desired distance BX. To avoid such a conflict a minimal acceleration is introduced:

```
bmin = -8 -2*car->accel+0.5*sqrt(v);
```

Another possibility is (and is used for instance at the PELOPS project at ika Aachen) not to aim the distance BX, but another distance which lies inside the following regime. Hence, before the problem of the singularity can occur, the regime has already changed and the formula is no longer valid.

Another feature of the Wiedemann model is to put into account, that the driver can better estimate the acceleration of his predecessor during the approach process. Wiedemann introduced a counter R, which is increased every time the program jumps into the routine "approach", "follow" or "emergency". In the routine "full_speed" this counter is set back to zero. This counter is used to correct the value of the acceleration in the approach routine:

```
b(i)=b(i)+(1-car->estimate)*(1-2*noise)/R(i);
```

If R increases, the second term gets smaller.

In the original Wiedemann paper a timestep of 1 second was used. To put the reaction time into account, he spent some effort and did some suspicious manipulation of the acceleration value with random numbers. In this simulation a smaller stepsize can be chosen (and is advised to choose). The values of the acceleration in the range of the reaction time have to be stored and can then be used for calculating the acceleration the reaction time later.

3.4 Routine emergency

This routine is almost a one to one copy of the routine approach. The only difference is the formula for the acceleration:

```
b(i,t) = 0.5*DV*DV/(AX-DX) + b(j,t-reaction_time);
```

BX swapped with AX. Because DX should be greater than AX all the time, the acceleration is negative all the time.

4 Results

Figure 3 shows the velocity distribution for three cars in a row. The first car accelerates until it has reached its desired velocity. Then this velocity is kept constant. The next car

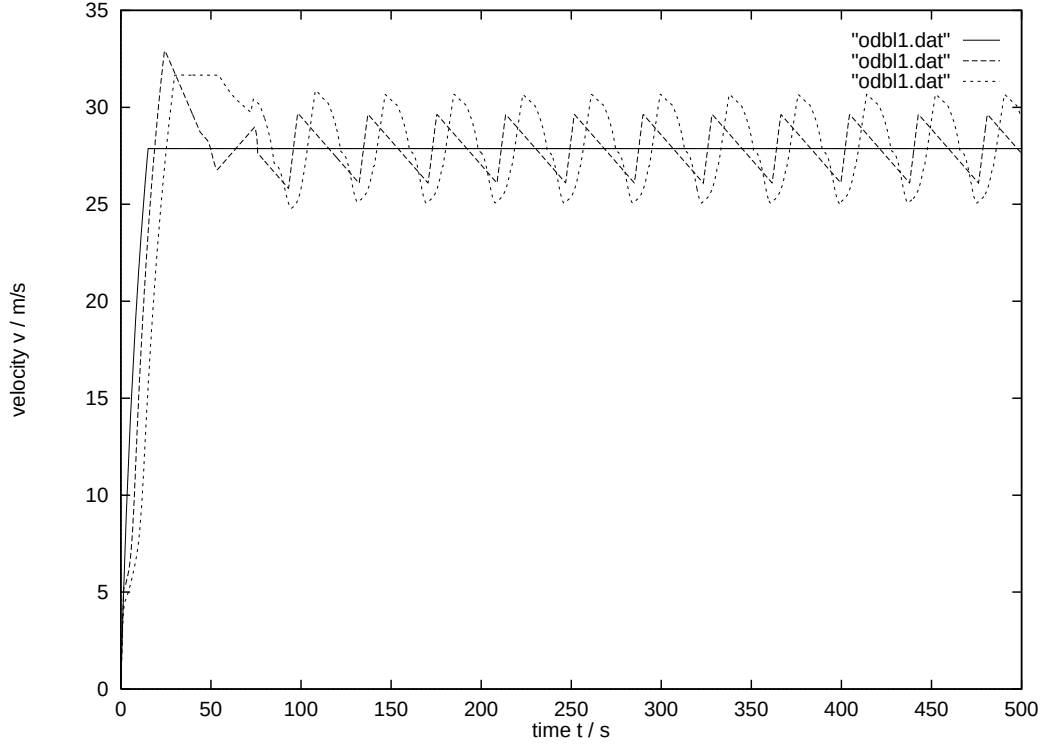


Figure 3: A small queue. Shown is velocity over time

would prefer to go faster than the first car. Hence, it has to adjust its velocity to that of the first car. It accelerates very strong (large slope in the diagram), comes to near to the predecessor and has to brake. When the distance is large enough it will accelerate again. The third car must now adjust its velocity to that oscillating motion of the second car. The graph is also shown in figure 3. Figure 4 shows the $dv-dx$ -diagram for the second car. The start is at $v=0\text{m/s}$ and $x=20\text{m}$. The first car started with $v=0\text{m/s}$ as well. The first car begins to accelerate (dv is negative in the diagram). When the distance is large enough, the second car begins to accelerate as well. At roughly 100m this curve has reached a maximum and this car is approaching its predecessor. After a certain time it looks like the second car is moving on a closed orbit and is oscillating between 30m and 50m compared to the leading car.

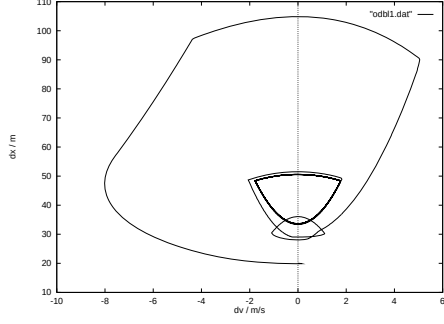


Figure 4: Second car

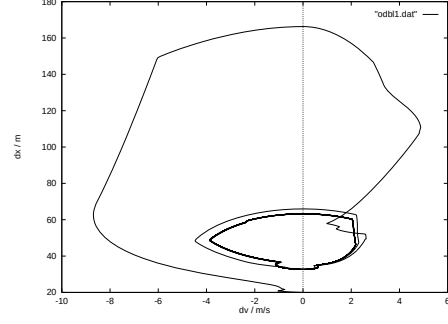


Figure 5: Third car

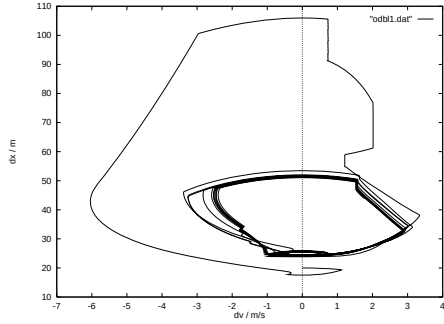


Figure 6: Forth car

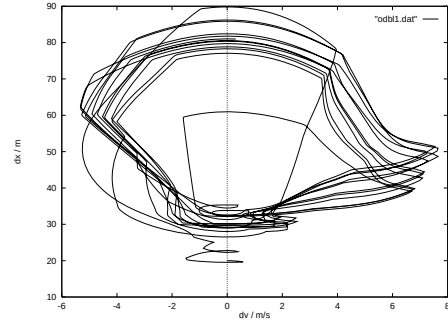


Figure 7: Fifth car

The dv - dx -diagram of the third car shows a bit more distortion, but the orbit looks still closed. This changes at the fourth car and surely at the fifth car. One still has to keep in mind, that the cars have to adjust their motion to the oscillating motion of their predecessor. Small disturbances will propagate back in the queue and will possibly amplify. The dv - dx -diagram of the fifth car has nothing much in common with the first car. The orbit is no longer closed and the position after the next timestep can not be predicted with good accuracy. All this results look like as if they describe chaotic behaviour. The motion of the last cars seems to be unpredictable (see figure 8).

Another sign for chaotic behaviour is the sensitivity of the system on initial parameters. This can be observed as well. This can be seen on the small queue with five cars from figure 4 to 7. The starting position of the first car was changed from 80m to 79m. The orbits in dv - dx space look for the first cars quite similar. They are all aiming roughly the same orbit. But it is getting worse the more cars are involved. In figure 9 the dv - dx -diagrams for both cases (starting position of the first car 80m and 79m) are compared.

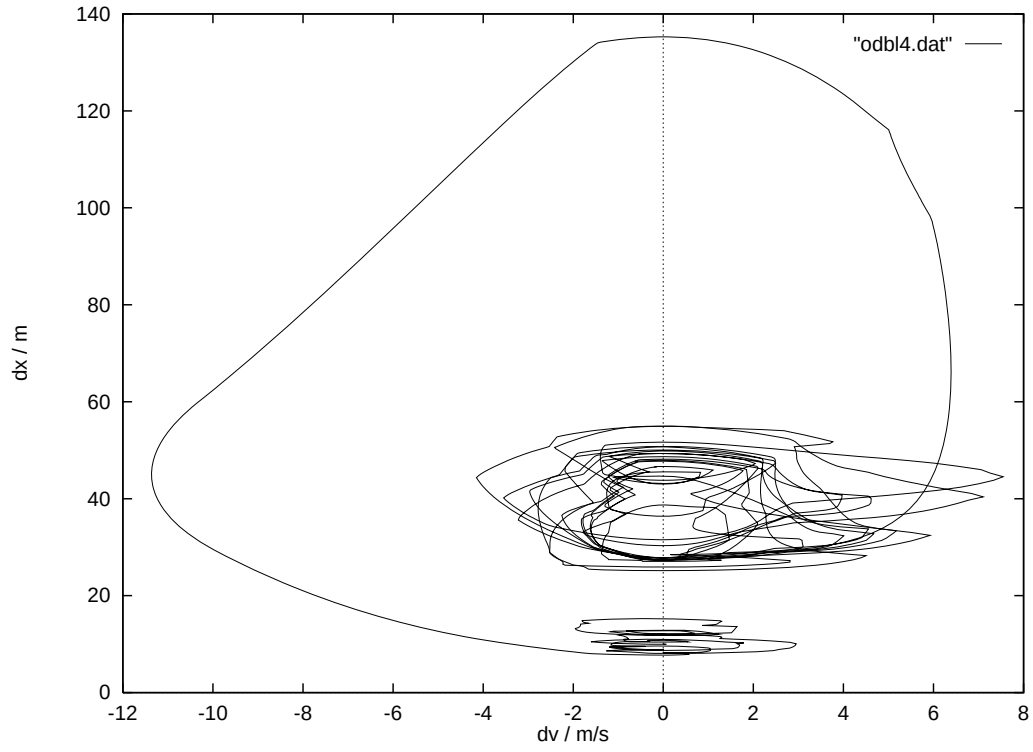


Figure 8: The sixth car in another queue

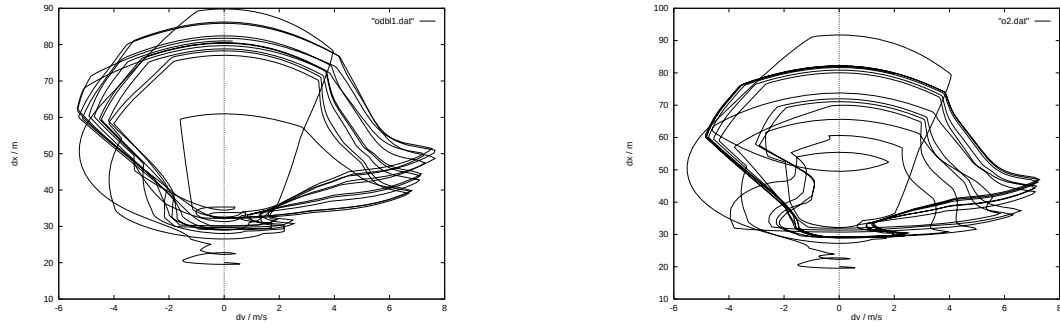


Figure 9: Fifth car for different starting positions of the first car: 80m and 79m.

How would all this look like if the reaction time is set to zero and therefore the delay is put out of the system? The drivers can steer their cars more accurately. Change in the situation are realised instantaneously and the right reaction is chosen. The program allows to simulate this. The reaction time has just to be set to zero. The results show the expected behaviour. Figure 10 shows again the fifth car in the queue, but now with no reaction time. One can see, that the distortions are smaller. The orbit has got a thinner shape.

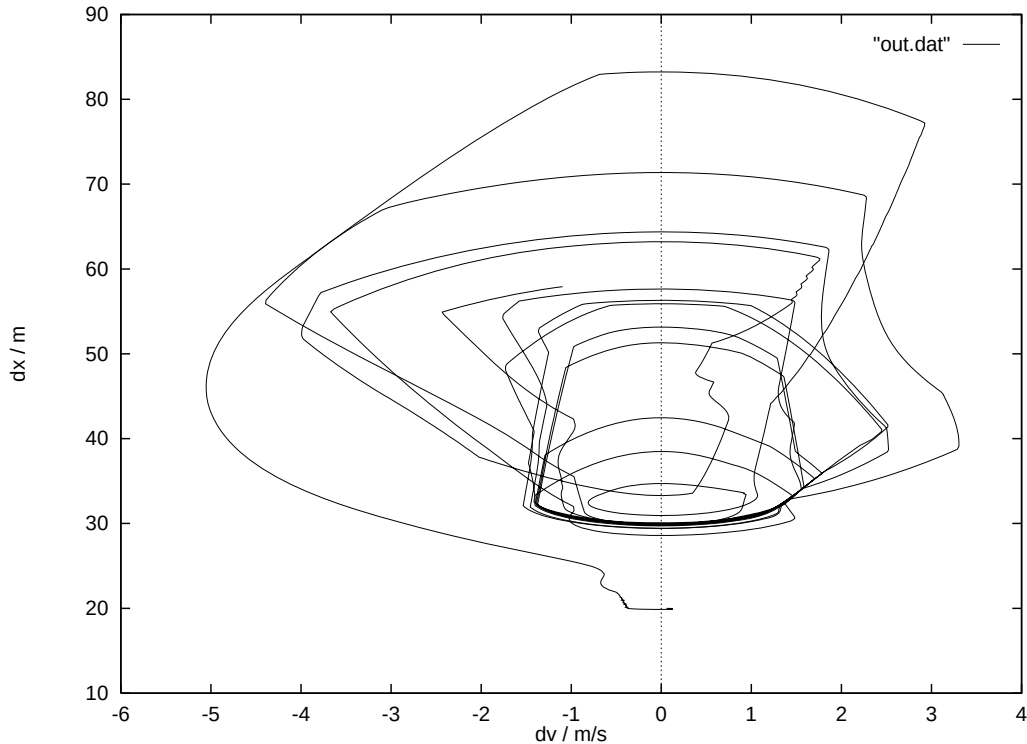


Figure 10: The fifth car with no reaction time

5 Outlook

There are a lot of things, which can be examined. To understand the behaviour of the cars better, it is useful to have something like a poincaré map. This can be done by inventing an oscillating leading car. If one forces the first car to oscillate with a constant period, one could examine the values of the velocity at a given phase.

Another point of interest is the choice of algorithm. The Euler method makes very small stepsizes unavoidable. An algorithm of higher order allows larger stepsizes and

minimizes therefore the computation time. It should be possible to implement a Runge Kutta algorithm. The problem is again the delay. The values of the acceleration in between are quite a problem. One might need to interpolate between different values.

If a traffic jam occurs, the Wiedemann model is in trouble. All calculated parameters are velocity dependent. In the case of a jam, the whole system shrinks together to a point (see figure 2). The model does not produce correct results in that case, because the program jumps from one regime to another regime after each timestep. To avoid this conflict, I think an extra rule is needed. If the velocity difference and the distance to the next car is too small, the new rule should be active. For example, set the acceleration to exactly the same value as the predecessor. At very small distances this should be okay, but there might be problems with the start again.

The actual program is only able to simulate one lane traffic. The Wiedemann model does not support lane changes or two lane traffic at all. However, it is possible to mix this model with a lane change model to simulate motorway traffic.

The Wiedemann model is described by a lot of small rules. It should be possible to use a more abstract model by inventing a potential between the cars which produces similar results. This potential should be velocity and position dependent. The computation time could be reduced by this procedure. Critical is again the delay in the system. This should be included as well.

6 Appendix

A Handling of the program

The program and the source code can be found in the folder

`~stebens/src/wiedemann.`

There can be found 2 programs: "wiedbl" and "wieflt". The first one calculates with double accuracy, whereas the second has got only float accuracy. 3 options can be handed to the program:

- -w filename

With this option the properties of all cars can be written to a file named by filename. They can then be changed (for instance to examine different behaviour at slightly different initial conditions).

- -r filename

With this option the properties of the cars can be read in from the file called filename. Usually this is a file which was saved with the -w option. Attention! The timestep and the reaction time have to be identical! Otherwise, the data is not read in correctly.

- -o filename

This option specifies the name of the output file. The default name is "out.dat".

The structure of the output file is as follows:

1 st row	time	
2 nd row	DX	
3 rd row	DV	First car
4 th row	x	
5 th row	v	
6 th row	DX	
7 th row	DV	Second car
:	:	

In the file "input.dat" one can choose the following parameters (the numbers are just example values):

number of cars	:	5
length of the road	[m]:	100
mean length of a car	[m]:	4.5
interaction range	[m]:	150
timestep	[s]:	0.01
calculation time	[s]:	500
reaction time in multiple of dt	:	100
desired velocity	[km/h]:	120
standard deviation of this	[km/h]:	20
noise -1=random else value	:	0

Note that the reaction time times the timestep yields to the reaction time in seconds, in this case 1 second.